Spécification OpenAPI — API FileFlow

```yaml
openapi: 3.1.0
info:
  title: API FileFlow
  version: 1.1.0
  summary: API REST de la plateforme de gestion de fichiers auto-hébergée FileFlow.
  description: >
    API de FileFlow, plateforme de gestion et de partage de fichiers auto-hébergée.


    L'API expose la gestion des fichiers (navigation, téléversement, versions,
    corbeille, recherche), le partage (liens publics, dossiers partagés, partages
    internes entre utilisateurs, demandes de téléversement), l'authentification
    (mot de passe + double authentification TOTP) et l'administration
    (utilisateurs, groupes, journal d'audit).


    **Conventions générales**

    - Toutes les routes sont préfixées par `/api`.

    - Les erreurs sont retournées avec le code HTTP approprié et un corps JSON
      `{ "error": "message" }`.

    - Les chemins de fichiers sont des chemins virtuels relatifs à la racine des
      données, exprimés avec `/` comme séparateur (ex. `/dossier/fichier.txt`).

    - L'authentification s'appuie sur un cookie de session `fileflow_token`
      (recommandé) ou sur un en-tête `Authorization: Bearer <jwt>`.
  contact:
    name: Driss OUAAZIZ — Software Architect
  license:
    name: Propriétaire — Driss OUAAZIZ

servers:
  - url: /api
    description: Chemin de base de l'API (même origine que l'application)

tags:
  - name: Authentification
    description: Connexion, déconnexion, session courante et double authentification
(2FA).
  - name: Fichiers
    description: Navigation, téléversement, téléchargement, prévisualisation et
opérations sur les fichiers.
  - name: Téléversement résumable
    description: Téléversement fragmenté et reprenable pour les fichiers volumineux.
  - name: Corbeille et versions
    description: Corbeille, restauration, versions de fichiers et métadonnées
(tags/collections).
  - name: Partage
    description: Liens publics, dossiers partagés, partages internes, favoris et
```

```
demandes de téléversement.
    - name: Public
      description: Points d'accès non authentifiés (liens de partage, pages de
téléversement public, sonde de santé).
    - name: Administration
      description: Gestion des utilisateurs, des groupes et du journal d'audit
(réservé aux administrateurs).

  security:
    - cookieAuth: []
    - bearerAuth: []

  paths:
    # ──────────────────────────── Authentification ────────────────────────────
    /auth/login:
      post:
        tags: [Authentification]
        summary: Authentifier un utilisateur
        description: >
          Vérifie les identifiants. Si la double authentification est activée pour
          le compte, retourne `{ "requires_2fa": true }` et dépose un cookie de
          session temporaire (`fileflow_2fa_pending`, valable 5 minutes) ; l'appelant
          doit ensuite appeler `/auth/2fa/verify`. Sinon, dépose le cookie de session
          `fileflow_token` et retourne le profil de l'utilisateur.


          Verrouillage anti-force-brute (`429`, 15 minutes) à trois niveaux : par
          adresse IP (5 échecs), par couple utilisateur + IP (5 échecs) et par
          compte toutes IP confondues (50 échecs). Le compteur par compte n'est pas
          remis à zéro par une connexion réussie ; il expire seul 15 minutes après
          le dernier échec.
        security: []
        requestBody:
          required: true
          content:
            application/json:
              schema: { $ref: '#/components/schemas/LoginRequest' }
        responses:
          '200':
            description: Connexion réussie ou challenge 2FA requis.
            content:
              application/json:
                schema:
                  oneOf:
                    - $ref: '#/components/schemas/UserInfo'
                    - $ref: '#/components/schemas/TwoFactorChallenge'
          '401': { $ref: '#/components/responses/Unauthorized' }
          '429': { $ref: '#/components/responses/TooManyRequests' }

    /auth/logout:
      post:
        tags: [Authentification]
        summary: Déconnecter la session courante
        description: >
          Ne requiert pas d'authentification valide : la déconnexion réussit même
          avec un jeton absent, expiré ou révoqué. Efface systématiquement les deux
```

```
        cookies (`fileflow_token` et `fileflow_2fa_pending`). L'événement
        `logout` n'est journalisé que si le jeton présenté est valide.
      security: []
      responses:
        '200':
          description: Déconnexion effectuée ; cookies de session expirés.
          headers:
            Set-Cookie:
              description: Expiration de `fileflow_token` et `fileflow_2fa_pending`
(`Max-Age=0`).
              schema: { type: string }

  /auth/logout-all:
    post:
      tags: [Authentification]
      summary: Déconnecter toutes les sessions
      description: >
        Incrémente la version de jeton de l'utilisateur, invalidant immédiatement
        tous les jetons JWT en circulation (tous les appareils).
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /auth/me:
    get:
      tags: [Authentification]
      summary: Profil de la session courante
      responses:
        '200':
          description: Profil de l'utilisateur authentifié.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/UserInfo' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /auth/2fa/setup:
    post:
      tags: [Authentification]
      summary: Initier l'activation de la 2FA
      description: >
        Génère un secret TOTP (non encore confirmé) et retourne l'URI
        `otpauth://` à afficher sous forme de QR code. L'activation n'est effective
        qu'après appel de `/auth/2fa/confirm`. Refusé (`400`) si la 2FA est déjà
        active : un secret actif n'est jamais remplacé ; pour en changer, il faut
        d'abord désactiver la 2FA (mot de passe et code exigés).
      responses:
        '200':
          description: Secret TOTP généré.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/TotpSetupResponse' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /auth/2fa/confirm:
    post:
```

```yaml
      tags: [Authentification]
      summary: Confirmer et activer la 2FA
      description: >
        Vérifie un premier code TOTP puis active la double authentification.
        Retourne les codes de récupération (à usage unique) — affichés une seule
fois.

        Refusé (`400`) si la 2FA est déjà active ou si aucune initialisation n'est
        en cours. En cas de succès, la session tourne : la version de jeton est
        incrémentée (toutes les autres sessions sont révoquées) et un nouveau
        cookie `fileflow_token` est déposé pour l'appelant.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TotpCode' }
      responses:
        '200':
          description: 2FA activée ; codes de récupération retournés.
          headers:
            Set-Cookie:
              description: Nouveau cookie de session `fileflow_token`.
              schema: { type: string }
          content:
            application/json:
              schema: { $ref: '#/components/schemas/TotpConfirmResponse' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /auth/2fa/verify:
    post:
      tags: [Authentification]
      summary: Vérifier le code TOTP (étape de connexion)
      description: >
        Consomme le cookie de session temporaire déposé par `/auth/login`, vérifie
        le code TOTP et, en cas de succès, dépose le cookie de session complet.
        Point d'accès protégé par limitation anti-force-brute.
      security:
        - pendingCookieAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TotpCode' }
      responses:
        '200':
          description: Vérification réussie ; session ouverte.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/UserInfo' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '429': { $ref: '#/components/responses/TooManyRequests' }

  /auth/2fa/verify-recovery:
    post:
      tags: [Authentification]
```

```yaml
    summary: Vérifier un code de récupération (étape de connexion)
    description: >
      Alternative à `/auth/2fa/verify` : consomme un code de récupération à usage
      unique. Protégé par la même limitation anti-force-brute.
    security:
      - pendingCookieAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/TotpRecoveryRequest' }
    responses:
      '200':
        description: Vérification réussie ; session ouverte.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/UserInfo' }
      '400': { $ref: '#/components/responses/BadRequest' }
      '401': { $ref: '#/components/responses/Unauthorized' }
      '429': { $ref: '#/components/responses/TooManyRequests' }

  /auth/2fa/disable:
    post:
      tags: [Authentification]
      summary: Désactiver la 2FA
      description: >
        Requiert le mot de passe courant **et** un second facteur (`code` : code
        TOTP courant à 6 chiffres, ou code de récupération, alors consommé). Une
        session volée seule, ou un mot de passe volé seul, ne suffit pas. Un échec
        renvoie un message unique (`400`) qui ne révèle pas lequel des deux est
        erroné, et compte dans le verrouillage 2FA de l'utilisateur (`429` après
        5 échecs, 15 minutes). En cas de succès, la session tourne : les autres
        sessions sont révoquées et un nouveau cookie `fileflow_token` est déposé.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TotpStepUpRequest' }
      responses:
        '204':
          description: 2FA désactivée ; nouveau cookie de session déposé.
          headers:
            Set-Cookie:
              description: Nouveau cookie de session `fileflow_token`.
              schema: { type: string }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '429': { $ref: '#/components/responses/TooManyRequests' }

  /auth/2fa/recovery-codes/regenerate:
    post:
      tags: [Authentification]
      summary: Régénérer les codes de récupération
      description: >
        Invalide les anciens codes et en génère de nouveaux. Requiert le mot de
        passe courant **et** un second facteur (`code` : code TOTP courant ou code
```

```
      de récupération). Mêmes règles d'échec et de verrouillage que
      `/auth/2fa/disable` (`400` à message unique, `429` après 5 échecs).
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/TotpStepUpRequest' }
    responses:
      '200':
        description: Nouveaux codes de récupération.
        content:
          application/json:
            schema:
              type: array
              items: { type: string }
      '400': { $ref: '#/components/responses/BadRequest' }
      '401': { $ref: '#/components/responses/Unauthorized' }
      '429': { $ref: '#/components/responses/TooManyRequests' }

# ──────────────────────────────── Fichiers ────────────────────────────────
/files:
  get:
    tags: [Fichiers]
    summary: Lister le contenu d'un répertoire
    parameters:
      - $ref: '#/components/parameters/PathQuery'
    responses:
      '200':
        description: Contenu du répertoire.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/DirectoryListing' }
      '401': { $ref: '#/components/responses/Unauthorized' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }

/files/download:
  get:
    tags: [Fichiers]
    summary: Télécharger un fichier
    description: >
      Diffuse le fichier en pièce jointe. Supporte les requêtes partielles
      (en-tête `Range`) : répond `206 Partial Content` pour une plage valide,
      `416` pour une plage non satisfiable.
    parameters:
      - $ref: '#/components/parameters/PathQueryRequired'
      - $ref: '#/components/parameters/RangeHeader'
    responses:
      '200':
        description: Contenu complet du fichier.
        content:
          application/octet-stream:
            schema: { type: string, format: binary }
      '206': { description: Contenu partiel (réponse à une requête `Range`). }
      '416': { description: Plage demandée non satisfiable. }
      '401': { $ref: '#/components/responses/Unauthorized' }
```

```
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }

  /files/download-bulk:
    get:
      tags: [Fichiers]
      summary: Télécharger plusieurs éléments en archive ZIP
      description: >
        Assemble les chemins demandés dans une archive ZIP. Le nombre de
        téléversements simultanés est borné côté serveur ; la taille cumulée est
        plafonnée.
      parameters:
        - name: paths
          in: query
          required: true
          description: Répété une fois par élément à inclure.
          schema:
            type: array
            items: { type: string }
          style: form
          explode: true
      responses:
        '200':
          description: Archive ZIP.
          content:
            application/zip:
              schema: { type: string, format: binary }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '403': { $ref: '#/components/responses/Forbidden' }

  /files/preview:
    get:
      tags: [Fichiers]
      summary: Prévisualiser un fichier (rendu inline)
      description: >
        Diffuse le fichier pour affichage dans le navigateur. Les types de contenu
        actif (HTML, SVG, XML…) sont volontairement servis en `text/plain` afin de
        neutraliser tout risque de script stocké. Supporte l'en-tête `Range`.
      parameters:
        - $ref: '#/components/parameters/PathQueryRequired'
        - $ref: '#/components/parameters/RangeHeader'
      responses:
        '200':
          description: Contenu du fichier pour prévisualisation.
          content:
            application/octet-stream:
              schema: { type: string, format: binary }
        '206': { description: Contenu partiel. }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /files/upload:
    post:
      tags: [Fichiers]
```

```yaml
    summary: Téléverser un ou plusieurs fichiers
    description: >
      Téléversement multipart classique vers le répertoire cible. Le quota de
      stockage et la liste noire d'extensions sont appliqués au fil de l'eau ;
      un fichier existant est archivé en tant que version avant remplacement.
      Pour les fichiers volumineux, préférer le téléversement résumable.
    parameters:
      - $ref: '#/components/parameters/PathQuery'
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            properties:
              files:
                type: array
                items: { type: string, format: binary }
    responses:
      '200':
        description: Fichiers téléversés.
        content:
          application/json:
            schema:
              type: object
              properties:
                uploaded: { type: array, items: { type: string } }
                count: { type: integer }
      '400': { $ref: '#/components/responses/BadRequest' }
      '401': { $ref: '#/components/responses/Unauthorized' }
      '403': { $ref: '#/components/responses/Forbidden' }

  /files/mkdir:
    post:
      tags: [Fichiers]
      summary: Créer un répertoire
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/MkdirRequest' }
      responses:
        '200':
          description: Répertoire créé.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/PathNameResult' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }

  /files/rename:
    post:
      tags: [Fichiers]
      summary: Renommer un fichier ou un répertoire
      requestBody:
        required: true
```

```yaml
        content:
          application/json:
            schema: { $ref: '#/components/schemas/RenameRequest' }
      responses:
        '200':
          description: Élément renommé.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/PathNameResult' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /files/delete:
    post:
      tags: [Fichiers]
      summary: Supprimer des éléments (mise en corbeille)
      description: Déplace les éléments vers la corbeille plutôt qu'une suppression
définitive.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/PathsRequest' }
      responses:
        '200':
          description: Éléments mis en corbeille.
          content:
            application/json:
              schema:
                type: object
                properties:
                  deleted: { type: array, items: { type: string } }
                  count: { type: integer }
        '403': { $ref: '#/components/responses/Forbidden' }

  /files/move:
    post:
      tags: [Fichiers]
      summary: Déplacer des éléments
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/MoveRequest' }
      responses:
        '200':
          description: Éléments déplacés.
          content:
            application/json:
              schema:
                type: object
                properties:
                  moved: { type: array, items: { type: string } }
                  count: { type: integer }
        '400': { $ref: '#/components/responses/BadRequest' }
```

```yaml
        '403': { $ref: '#/components/responses/Forbidden' }

  /files/search:
    get:
      tags: [Fichiers]
      summary: Rechercher des fichiers
      description: Recherche par nom, avec option de recherche plein texte sur les
fichiers texte.
      parameters:
        - name: q
          in: query
          required: true
          description: Terme de recherche (2 caractères minimum).
          schema: { type: string, minLength: 2 }
        - $ref: '#/components/parameters/PathQuery'
        - name: full_text
          in: query
          description: Active la recherche dans le contenu des fichiers texte.
          schema: { type: boolean }
      responses:
        '200':
          description: Résultats de recherche.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/FileEntry' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }

  # ──────────────────────── Téléversement résumable ────────────────────────
  /uploads/init:
    post:
      tags: [Téléversement résumable]
      summary: Initialiser un téléversement résumable
      description: Valide l'accès, la liste noire d'extensions et le quota, puis
crée une session.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/InitUploadRequest' }
      responses:
        '200':
          description: Session créée.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/InitUploadResponse' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }

  /uploads/chunk:
    put:
      tags: [Téléversement résumable]
      summary: Envoyer un fragment
      description: >
```

```
      Ajoute un fragment à l'offset attendu. En cas de décalage d'offset, répond
      `409 Conflict` avec l'offset attendu. Taille de fragment plafonnée à 32 Mio.
    parameters:
      - name: upload_id
        in: query
        required: true
        schema: { type: string }
      - name: offset
        in: query
        required: true
        schema: { type: integer, format: int64 }
    requestBody:
      required: true
      content:
        application/octet-stream:
          schema: { type: string, format: binary }
    responses:
      '200':
        description: Fragment reçu.
        content:
          application/json:
            schema:
              type: object
              properties:
                offset: { type: integer, format: int64 }
                total: { type: integer, format: int64 }
      '400': { $ref: '#/components/responses/BadRequest' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }
      '409': { $ref: '#/components/responses/Conflict' }

  /uploads/status:
    get:
      tags: [Téléversement résumable]
      summary: Consulter l'avancement d'une session
      parameters:
        - name: upload_id
          in: query
          required: true
          schema: { type: string }
      responses:
        '200':
          description: Offset reçu et taille totale.
          content:
            application/json:
              schema:
                type: object
                properties:
                  offset: { type: integer, format: int64 }
                  total: { type: integer, format: int64 }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /uploads/finish:
    post:
      tags: [Téléversement résumable]
```

```yaml
      summary: Finaliser un téléversement résumable
      description: Vérifie le nombre d'octets, archive une éventuelle version
existante et met le fichier en place.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/UploadIdRequest' }
      responses:
        '200':
          description: Fichier finalisé.
          content:
            application/json:
              schema:
                type: object
                properties:
                  uploaded: { type: string }
                  size: { type: integer, format: int64 }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /uploads/abort:
    post:
      tags: [Téléversement résumable]
      summary: Abandonner un téléversement résumable
      parameters:
        - name: upload_id
          in: query
          required: true
          schema: { type: string }
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '403': { $ref: '#/components/responses/Forbidden' }

  # ──────────────────────── Corbeille / versions / tags ──────────────────────
  /trash:
    get:
      tags: [Corbeille et versions]
      summary: Lister la corbeille
      description: Retourne les éléments de l'utilisateur (tous les éléments pour un
administrateur). Les éléments de plus de 30 jours sont purgés.
      responses:
        '200':
          description: Éléments en corbeille.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/TrashEntry' }
        '401': { $ref: '#/components/responses/Unauthorized' }

  /trash/restore:
    post:
      tags: [Corbeille et versions]
      summary: Restaurer un élément de la corbeille
```

```yaml
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/RestoreRequest' }
      responses:
        '200':
          description: Élément restauré.
          content:
            application/json:
              schema: { type: object, properties: { restored: { type: boolean } } }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /trash/empty:
    post:
      tags: [Corbeille et versions]
      summary: Vider la corbeille
      description: Vide la corbeille de l'utilisateur (ou l'intégralité pour un
administrateur).
      responses:
        '200':
          description: Corbeille vidée.
          content:
            application/json:
              schema: { type: object, properties: { emptied: { type: boolean } } }

  /versions:
    get:
      tags: [Corbeille et versions]
      summary: Lister les versions d'un fichier
      parameters:
        - $ref: '#/components/parameters/PathQueryRequired'
      responses:
        '200':
          description: Versions disponibles (plus récentes d'abord).
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/VersionEntry' }
        '403': { $ref: '#/components/responses/Forbidden' }

  /versions/restore:
    post:
      tags: [Corbeille et versions]
      summary: Restaurer une version de fichier
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/VersionRestoreRequest' }
      responses:
        '200':
          description: Version restaurée.
```

```
        content:
          application/json:
            schema: { type: object, properties: { restored: { type: boolean } } }
      '400': { $ref: '#/components/responses/BadRequest' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }

  /metadata/tags:
    get:
      tags: [Corbeille et versions]
      summary: Lire les tags d'un fichier
      parameters:
        - $ref: '#/components/parameters/PathQueryRequired'
      responses:
        '200':
          description: Métadonnées du fichier.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/FileMetadata' }
        '403': { $ref: '#/components/responses/Forbidden' }
    post:
      tags: [Corbeille et versions]
      summary: Mettre à jour les tags d'un fichier
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/TagUpdateRequest' }
      responses:
        '200':
          description: Tags mis à jour.
          content:
            application/json:
              schema: { type: object, properties: { success: { type: boolean } } }
        '403': { $ref: '#/components/responses/Forbidden' }

  /metadata/tags/bulk:
    post:
      tags: [Corbeille et versions]
      summary: Mettre à jour les tags de plusieurs fichiers
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/BulkTagUpdateRequest' }
      responses:
        '200':
          description: Tags mis à jour en masse.
          content:
            application/json:
              schema:
                type: object
                properties:
                  success: { type: boolean }
                  count: { type: integer }
        '403': { $ref: '#/components/responses/Forbidden' }
```

```yaml
  /metadata/search:
    get:
      tags: [Corbeille et versions]
      summary: Rechercher des fichiers par tag
      parameters:
        - name: q
          in: query
          required: true
          description: Tag recherché (le préfixe « # » est ignoré). 2 caractères minimum.
          schema: { type: string, minLength: 2 }
      responses:
        '200':
          description: Fichiers correspondants.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/FileEntry' }
        '400': { $ref: '#/components/responses/BadRequest' }

  # ──────────────────────────────── Partage ────────────────────────────────
  /shares:
    get:
      tags: [Partage]
      summary: Lister les liens de partage
      description: Un utilisateur voit ses propres liens ; un administrateur voit tous les liens.
      responses:
        '200':
          description: Liens de partage.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/ShareLinkView' }
        '401': { $ref: '#/components/responses/Unauthorized' }
    post:
      tags: [Partage]
      summary: Créer un lien de partage public
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CreateShareRequest' }
      responses:
        '200':
          description: Lien créé.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/ShareLinkView' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /shares/{token}:
```

```yaml
    delete:
      tags: [Partage]
      summary: Supprimer un lien de partage
      parameters:
        - $ref: '#/components/parameters/TokenPath'
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /shared-folders:
    get:
      tags: [Partage]
      summary: Lister les dossiers partagés
      responses:
        '200':
          description: Dossiers partagés accessibles.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/SharedFolder' }
    post:
      tags: [Partage]
      summary: Créer un dossier partagé
      description: >
        Exige que le créateur dispose de l'accès en lecture **en propre** sur le
        dossier (rôle, permissions et groupes ; un partage interne ou un autre
        dossier partagé ne suffit pas), sinon `403`. Les droits des membres sont
        dérivés de ceux du créateur et revérifiés à chaque accès : un membre
        n'obtient jamais plus que le créateur, et retirer ses droits au créateur
        les retire aux membres.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CreateSharedFolderRequest' }
      responses:
        '200':
          description: Dossier partagé créé.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/SharedFolder' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /shared-folders/{id}:
    get:
      tags: [Partage]
      summary: Détails d'un dossier partagé
      parameters:
        - $ref: '#/components/parameters/IdPath'
      responses:
        '200':
          description: Dossier partagé.
```

```
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/SharedFolder' }
              '403': { $ref: '#/components/responses/Forbidden' }
              '404': { $ref: '#/components/responses/NotFound' }
          delete:
            tags: [Partage]
            summary: Supprimer un dossier partagé
            parameters:
              - $ref: '#/components/parameters/IdPath'
            responses:
              '204': { $ref: '#/components/responses/NoContent' }
              '403': { $ref: '#/components/responses/Forbidden' }
              '404': { $ref: '#/components/responses/NotFound' }

        /shared-folders/{id}/members:
          put:
            tags: [Partage]
            summary: Mettre à jour les membres d'un dossier partagé
            description: >
              Accorder l'écriture (`can_write: true`) à un membre exige que le créateur
              dispose lui-même de l'accès en écriture en propre sur le dossier, sinon
`403`.
            parameters:
              - $ref: '#/components/parameters/IdPath'
            requestBody:
              required: true
              content:
                application/json:
                  schema:
                    type: array
                    items: { $ref: '#/components/schemas/SharedFolderMember' }
            responses:
              '200':
                description: Membres mis à jour.
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/SharedFolder' }
              '403': { $ref: '#/components/responses/Forbidden' }
              '404': { $ref: '#/components/responses/NotFound' }

        /internal-shares:
          get:
            tags: [Partage]
            summary: Lister les partages internes
            description: Retourne les partages émis (`owned`) et reçus (`received`) par
l'utilisateur.
            responses:
              '200':
                description: Partages internes.
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/InternalSharesResponse' }
          post:
            tags: [Partage]
            summary: Créer un partage interne (d'utilisateur à utilisateur)
```

```yaml
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CreateInternalShareRequest' }
      responses:
        '200':
          description: Partage interne créé.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/InternalShare' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /internal-shares/{id}:
    delete:
      tags: [Partage]
      summary: Supprimer un partage interne
      description: Autorisé pour l'émetteur, le destinataire (refus) ou un
administrateur.
      parameters:
        - $ref: '#/components/parameters/IdPath'
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /shared-with-me:
    get:
      tags: [Partage]
      summary: Lister les éléments partagés avec moi
      responses:
        '200':
          description: Éléments reçus via partage interne.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/FileEntry' }

  /favorites:
    get:
      tags: [Partage]
      summary: Lister les favoris
      responses:
        '200':
          description: Favoris de l'utilisateur.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/Favorite' }
    post:
      tags: [Partage]
      summary: Ajouter un favori
```

```yaml
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/AddFavoriteRequest' }
    responses:
      '200':
        description: Favori ajouté.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Favorite' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }
  delete:
    tags: [Partage]
    summary: Retirer un favori
    parameters:
      - $ref: '#/components/parameters/PathQueryRequired'
    responses:
      '204': { $ref: '#/components/responses/NoContent' }

/upload-requests:
  get:
    tags: [Partage]
    summary: Lister les demandes de téléversement
    responses:
      '200':
        description: Demandes de téléversement.
        content:
          application/json:
            schema:
              type: array
              items: { $ref: '#/components/schemas/UploadRequestView' }
  post:
    tags: [Partage]
    summary: Créer une demande de téléversement public
    description: >
      Exige l'accès en écriture **en propre** sur le dossier cible (rôle,
      permissions, groupes) ; un accès obtenu via un partage interne ou un
      dossier partagé ne suffit pas (`403`). Le chemin est stocké sous forme
      canonique. La demande suit le dossier en cas de renommage ou de
      déplacement et est supprimée avec lui.
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/CreateUploadRequest' }
    responses:
      '200':
        description: Demande créée.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/UploadRequestView' }
      '400': { $ref: '#/components/responses/BadRequest' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }
```

```yaml
  /upload-requests/{token}:
    delete:
      tags: [Partage]
      summary: Supprimer une demande de téléversement
      parameters:
        - $ref: '#/components/parameters/TokenPath'
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  # ──────────────────────────────── Public ────────────────────────────────
  /s/{token}:
    get:
      tags: [Public]
      summary: Accéder à un lien de partage
      description: >
        Sert le fichier partagé, ou renvoie une page HTML de saisie de mot de passe
        si le lien est protégé. Retourne un `404` uniforme si le lien est expiré,
        a atteint sa limite de téléchargements, si son créateur est désactivé ou
        n'a plus l'accès en lecture au fichier (revérifié à chaque requête), ou si
        le fichier a été supprimé (le lien est alors supprimé avec lui ; il suit
        en revanche le fichier lors d'un renommage ou d'un déplacement).


        Plafond de téléchargements : la vérification et l'incrément sont
        atomiques (un téléchargement en échec est restitué). Un lien **avec**
        plafond ignore l'en-tête `Range` et sert toujours le fichier complet
        (`200`, jamais `206`), afin qu'un lien à usage unique ne puisse pas être
        récupéré par morceaux ; un lien sans plafond accepte `Range` (`206`).
      security: []
      parameters:
        - $ref: '#/components/parameters/TokenPath'
        - $ref: '#/components/parameters/RangeHeader'
      responses:
        '200':
          description: Fichier partagé, ou page HTML de mot de passe.
          content:
            application/octet-stream:
              schema: { type: string, format: binary }
            text/html:
              schema: { type: string }
        '206': { description: Contenu partiel (liens sans plafond de téléchargements
uniquement). }
        '404': { $ref: '#/components/responses/NotFound' }
    post:
      tags: [Public]
      summary: Soumettre le mot de passe d'un lien de partage
      description: >
        Les tentatives de mot de passe sont limitées sur deux clés, le lien et
        l'adresse IP du client : après 5 échecs, la clé concernée est verrouillée
        15 minutes et la requête reçoit `429` (vérifié avant tout calcul bcrypt).
        Un succès n'efface que le compteur du lien. Le fichier est servi en
        entier (pas de `Range`).
      security: []
```

```
    parameters:
      - $ref: '#/components/parameters/TokenPath'
    requestBody:
      required: true
      content:
        application/x-www-form-urlencoded:
          schema:
            type: object
            properties:
              pwd: { type: string }
    responses:
      '200':
        description: Fichier servi si le mot de passe est correct, sinon page HTML
avec erreur.
        content:
          application/octet-stream:
            schema: { type: string, format: binary }
          text/html:
            schema: { type: string }
      '404': { $ref: '#/components/responses/NotFound' }
      '429': { $ref: '#/components/responses/TooManyRequests' }

  /r/{token}:
    get:
      tags: [Public]
      summary: Afficher la page de téléversement public
      description: Retourne `404` si la demande est inconnue ou expirée.
      security: []
      parameters:
        - $ref: '#/components/parameters/TokenPath'
      responses:
        '200':
          description: Page HTML de téléversement.
          content:
            text/html:
              schema: { type: string }
        '404': { $ref: '#/components/responses/NotFound' }
    post:
      tags: [Public]
      summary: Déposer des fichiers via une demande de téléversement
      description: >
        Dépôt anonyme dans le dossier de la demande. Règles appliquées :


        - le champ `pwd` doit **précéder** les champs `files` dans le corps
          multipart : un fichier reçu avant la vérification du mot de passe est
          refusé (page HTML d'erreur) ;

        - tentatives de mot de passe limitées sur deux clés (lien et adresse IP) :
          5 échecs → verrouillage de 15 minutes, réponse `429` ;

        - taille cumulée plafonnée par `PUBLIC_UPLOAD_MAX_MB` (100 Mio par défaut,
          jamais au-delà de `MAX_UPLOAD_MB`), réponse `400` en cas de dépassement ;
          liste noire d'extensions et quota du créateur appliqués ;

        - aucun écrasement : si un fichier du même nom existe, le dépôt reçoit un
```

```
                nom numéroté (`rapport (1).txt`), réservé de façon atomique ; les
                fichiers temporaires sont supprimés en cas d'erreur ;

              - les droits d'écriture propres du créateur sont revérifiés à chaque
                dépôt (`404` s'il les a perdus).
            security: []
            parameters:
              - $ref: '#/components/parameters/TokenPath'
            requestBody:
              required: true
              content:
                multipart/form-data:
                  schema:
                    type: object
                    properties:
                      pwd:
                        type: string
                        description: Mot de passe de la demande ; doit être envoyé avant
les fichiers.
                      files:
                        type: array
                        items: { type: string, format: binary }
                  encoding:
                    files:
                      contentType: application/octet-stream
            responses:
              '200':
                description: Page HTML de confirmation (succès ou erreur).
                content:
                  text/html:
                    schema: { type: string }
              '400': { $ref: '#/components/responses/BadRequest' }
              '404': { $ref: '#/components/responses/NotFound' }
              '429': { $ref: '#/components/responses/TooManyRequests' }

        /health:
          get:
            tags: [Public]
            summary: Sonde de santé
            description: Point d'accès de liveness non authentifié, utilisé par les
orchestrateurs.
            security: []
            responses:
              '200':
                description: Service opérationnel.
                content:
                  application/json:
                    schema:
                      type: object
                      properties:
                        status: { type: string, example: ok }
                        version: { type: string, example: 1.0.0 }

        # ──────────────────────────── Administration ────────────────────────────
        /admin/users:
          get:
```

```yaml
    tags: [Administration]
    summary: Lister les utilisateurs
    responses:
      '200':
        description: Liste des utilisateurs.
        content:
          application/json:
            schema:
              type: array
              items: { $ref: '#/components/schemas/UserView' }
      '403': { $ref: '#/components/responses/Forbidden' }
  post:
    tags: [Administration]
    summary: Créer un utilisateur
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/CreateUserRequest' }
    responses:
      '201':
        description: Utilisateur créé.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/UserView' }
      '400': { $ref: '#/components/responses/BadRequest' }
      '403': { $ref: '#/components/responses/Forbidden' }

/admin/users/{username}:
  put:
    tags: [Administration]
    summary: Mettre à jour un utilisateur
    description: >
      Refusé (`400`) s'il s'agit de rétrograder ou de désactiver le dernier
      administrateur **actif** (un administrateur désactivé ne compte pas). Le
      contrôle est effectué sous le même verrou que la modification.
    parameters:
      - $ref: '#/components/parameters/UsernamePath'
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: '#/components/schemas/UpdateUserRequest' }
    responses:
      '200':
        description: Utilisateur mis à jour.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/UserView' }
      '400': { $ref: '#/components/responses/BadRequest' }
      '403': { $ref: '#/components/responses/Forbidden' }
      '404': { $ref: '#/components/responses/NotFound' }
  delete:
    tags: [Administration]
    summary: Supprimer un utilisateur
    description: >
```

```
        Refusé (`400`) pour son propre compte et pour le dernier administrateur
        actif (contrôle effectué sous le même verrou que la suppression).
      parameters:
        - $ref: '#/components/parameters/UsernamePath'
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /admin/users/{username}/password:
    put:
      tags: [Administration]
      summary: Réinitialiser le mot de passe d'un utilisateur
      parameters:
        - $ref: '#/components/parameters/UsernamePath'
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/ChangePasswordRequest' }
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '403': { $ref: '#/components/responses/Forbidden' }

  /admin/users/{username}/revoke-sessions:
    post:
      tags: [Administration]
      summary: Révoquer toutes les sessions d'un utilisateur
      parameters:
        - $ref: '#/components/parameters/UsernamePath'
      responses:
        '204': { $ref: '#/components/responses/NoContent' }
        '403': { $ref: '#/components/responses/Forbidden' }
        '404': { $ref: '#/components/responses/NotFound' }

  /admin/groups:
    get:
      tags: [Administration]
      summary: Lister les groupes
      responses:
        '200':
          description: Liste des groupes.
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/UserGroup' }
        '403': { $ref: '#/components/responses/Forbidden' }
    post:
      tags: [Administration]
      summary: Créer un groupe
      requestBody:
        required: true
        content:
```

```
                application/json:
                  schema: { $ref: '#/components/schemas/CreateGroupRequest' }
            responses:
              '201':
                description: Groupe créé.
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/UserGroup' }
              '403': { $ref: '#/components/responses/Forbidden' }

          /admin/groups/{id}:
            put:
              tags: [Administration]
              summary: Mettre à jour un groupe
              parameters:
                - $ref: '#/components/parameters/IdPath'
              requestBody:
                required: true
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/EditGroupRequest' }
              responses:
                '200':
                  description: Groupe mis à jour.
                  content:
                    application/json:
                      schema: { $ref: '#/components/schemas/UserGroup' }
                '403': { $ref: '#/components/responses/Forbidden' }
                '404': { $ref: '#/components/responses/NotFound' }
            delete:
              tags: [Administration]
              summary: Supprimer un groupe
              parameters:
                - $ref: '#/components/parameters/IdPath'
              responses:
                '204': { $ref: '#/components/responses/NoContent' }
                '403': { $ref: '#/components/responses/Forbidden' }
                '404': { $ref: '#/components/responses/NotFound' }

          /admin/audit:
            get:
              tags: [Administration]
              summary: Consulter le journal d'audit
              parameters:
                - { name: username, in: query, schema: { type: string } }
                - { name: action, in: query, schema: { type: string } }
                - { name: from, in: query, schema: { type: string, format: date-time } }
                - { name: to, in: query, schema: { type: string, format: date-time } }
                - { name: resource, in: query, schema: { type: string } }
                - { name: limit, in: query, schema: { type: integer, default: 500, maximum:
5000 } }
                - { name: offset, in: query, schema: { type: integer, default: 0 } }
              responses:
                '200':
                  description: Entrées d'audit (plus récentes d'abord).
                  content:
```

```
                    application/json:
                      schema:
                        type: array
                        items: { $ref: '#/components/schemas/AuditEntry' }
                '403': { $ref: '#/components/responses/Forbidden' }

          /admin/audit/export:
            get:
              tags: [Administration]
              summary: Exporter le journal d'audit en CSV
              parameters:
                - { name: username, in: query, schema: { type: string } }
                - { name: action, in: query, schema: { type: string } }
                - { name: from, in: query, schema: { type: string, format: date-time } }
                - { name: to, in: query, schema: { type: string, format: date-time } }
                - { name: resource, in: query, schema: { type: string } }
              responses:
                '200':
                  description: Fichier CSV.
                  content:
                    text/csv:
                      schema: { type: string }
                '403': { $ref: '#/components/responses/Forbidden' }

          /users/me/password:
            put:
              tags: [Administration]
              summary: Changer son propre mot de passe
              description: Requiert le mot de passe courant ; le nouveau doit en différer (8
caractères minimum).
              requestBody:
                required: true
                content:
                  application/json:
                    schema: { $ref: '#/components/schemas/ChangePasswordRequest' }
              responses:
                '204': { $ref: '#/components/responses/NoContent' }
                '400': { $ref: '#/components/responses/BadRequest' }
                '401': { $ref: '#/components/responses/Unauthorized' }

        components:
          securitySchemes:
            cookieAuth:
              type: apiKey
              in: cookie
              name: fileflow_token
              description: Cookie de session JWT (HttpOnly, SameSite=Strict, Secure). Déposé
à la connexion.
            bearerAuth:
              type: http
              scheme: bearer
              bearerFormat: JWT
              description: "Alternative au cookie — en-tête `Authorization: Bearer <jwt>`."
            pendingCookieAuth:
              type: apiKey
              in: cookie
```

```
            name: fileflow_2fa_pending
            description: Cookie temporaire (5 min) émis après l'étape mot de passe,
consommé par la vérification 2FA.

        parameters:
          PathQuery:
            name: path
            in: query
            description: Chemin virtuel (défaut « / »).
            schema: { type: string }
          PathQueryRequired:
            name: path
            in: query
            required: true
            description: Chemin virtuel de la ressource.
            schema: { type: string }
          RangeHeader:
            name: Range
            in: header
            description: Requête de plage d'octets (RFC 7233), ex. `bytes=0-1023`.
            schema: { type: string }
          TokenPath:
            name: token
            in: path
            required: true
            schema: { type: string }
          IdPath:
            name: id
            in: path
            required: true
            schema: { type: string }
          UsernamePath:
            name: username
            in: path
            required: true
            schema: { type: string }

        responses:
          NoContent:
            description: Opération réussie, sans contenu.
          BadRequest:
            description: Requête invalide.
            content:
              application/json:
                schema: { $ref: '#/components/schemas/Error' }
          Unauthorized:
            description: Authentification requise ou invalide.
            content:
              application/json:
                schema: { $ref: '#/components/schemas/Error' }
          Forbidden:
            description: Accès refusé (droits insuffisants).
            content:
              application/json:
                schema: { $ref: '#/components/schemas/Error' }
          NotFound:
```

```yaml
        description: Ressource introuvable.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      Conflict:
        description: Conflit (ex. offset de fragment inattendu).
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      TooManyRequests:
        description: >
          Trop de tentatives — verrouillage temporaire anti-force-brute (15 minutes).
          Les verrous sont tenus en mémoire et levés par un redémarrage du service.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      ServerError:
        description: Erreur interne du serveur.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

  schemas:
    Error:
      type: object
      description: Enveloppe d'erreur standard.
      required: [error]
      properties:
        error: { type: string, description: Message d'erreur lisible. }

    Permission:
      type: string
      enum: [file_read, file_write, file_delete, user_manage, group_manage,
view_audit]

    Role:
      type: string
      enum: [admin, manager, editor, viewer, user]

    UserStatus:
      type: string
      enum: [active, disabled]

    LoginRequest:
      type: object
      required: [username, password]
      properties:
        username: { type: string }
        password: { type: string, format: password }

    TwoFactorChallenge:
      type: object
      properties:
        requires_2fa: { type: boolean, const: true }

    UserInfo:
```

```yaml
    type: object
    description: Profil de la session courante.
    properties:
      username: { type: string }
      role: { type: string }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      totp_enabled: { type: boolean }

  UserMetadata:
    type: object
    properties:
      full_name: { type: [string, 'null'] }
      email: { type: [string, 'null'] }

  UserView:
    type: object
    description: Vue d'un utilisateur exposée à l'administration (sans secret).
    properties:
      username: { type: string }
      role: { $ref: '#/components/schemas/Role' }
      status: { $ref: '#/components/schemas/UserStatus' }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      groups:
        type: array
        items: { type: string }
      quota_gb: { type: [number, 'null'] }
      used_bytes: { type: integer, format: int64 }
      last_login: { type: [string, 'null'], format: date-time }
      metadata: { $ref: '#/components/schemas/UserMetadata' }
      created_at: { type: string, format: date-time }

  UserGroup:
    type: object
    properties:
      id: { type: string }
      name: { type: string }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      allowed_paths:
        type: array
        items: { type: string }

  CreateUserRequest:
    type: object
    required: [username, password]
    properties:
      username: { type: string, minLength: 1, maxLength: 64 }
      password: { type: string, minLength: 8, format: password }
      role: { $ref: '#/components/schemas/Role' }

  UpdateUserRequest:
```

```yaml
    type: object
    description: Champs optionnels ; seuls les champs fournis sont modifiés.
    properties:
      role: { $ref: '#/components/schemas/Role' }
      status: { $ref: '#/components/schemas/UserStatus' }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      groups:
        type: array
        items: { type: string }
      quota_gb: { type: [number, 'null'] }
      metadata: { $ref: '#/components/schemas/UserMetadata' }

  CreateGroupRequest:
    type: object
    required: [name, permissions, allowed_paths]
    properties:
      name: { type: string }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      allowed_paths:
        type: array
        items: { type: string }

  EditGroupRequest:
    type: object
    properties:
      name: { type: string }
      permissions:
        type: array
        items: { $ref: '#/components/schemas/Permission' }
      allowed_paths:
        type: array
        items: { type: string }

  ChangePasswordRequest:
    type: object
    required: [new_password]
    properties:
      new_password: { type: string, minLength: 8, format: password }
      current_password:
        type: [string, 'null']
        format: password
        description: Requis pour le changement de son propre mot de passe.

  FileEntry:
    type: object
    properties:
      name: { type: string }
      path: { type: string }
      is_dir: { type: boolean }
      size: { type: integer, format: int64 }
      modified: { type: [string, 'null'], format: date-time }
      extension: { type: [string, 'null'] }
```

```
    mime_type: { type: [string, 'null'] }

DirectoryListing:
  type: object
  properties:
    path: { type: string }
    parent: { type: [string, 'null'] }
    entries:
      type: array
      items: { $ref: '#/components/schemas/FileEntry' }

FileMetadata:
  type: object
  properties:
    tags:
      type: array
      items: { type: string }
    collections:
      type: array
      items: { type: string }

MkdirRequest:
  type: object
  required: [path, name]
  properties:
    path: { type: string }
    name: { type: string }

RenameRequest:
  type: object
  required: [old_path, new_name]
  properties:
    old_path: { type: string }
    new_name: { type: string }

PathsRequest:
  type: object
  required: [paths]
  properties:
    paths:
      type: array
      items: { type: string }

MoveRequest:
  type: object
  required: [paths, destination]
  properties:
    paths:
      type: array
      items: { type: string }
    destination: { type: string }

PathNameResult:
  type: object
  properties:
    path: { type: string }
```

```
    name: { type: string }

TagUpdateRequest:
  type: object
  required: [path]
  properties:
    path: { type: string }
    tags: { type: [array, 'null'], items: { type: string } }
    collections: { type: [array, 'null'], items: { type: string } }

BulkTagUpdateRequest:
  type: object
  required: [paths, tags, collections]
  properties:
    paths: { type: array, items: { type: string } }
    tags: { type: array, items: { type: string } }
    collections: { type: array, items: { type: string } }

TrashEntry:
  type: object
  properties:
    id: { type: string }
    original_path: { type: string }
    deleted_at: { type: string, format: date-time }
    username: { type: string }
    size: { type: integer, format: int64 }

VersionEntry:
  type: object
  properties:
    path: { type: string }
    version_id: { type: string }
    timestamp: { type: string, format: date-time }
    size: { type: integer, format: int64 }

RestoreRequest:
  type: object
  required: [id]
  properties:
    id: { type: string }

VersionRestoreRequest:
  type: object
  required: [path, version_id]
  properties:
    path: { type: string }
    version_id: { type: string }

InitUploadRequest:
  type: object
  required: [path, filename, size]
  properties:
    path: { type: string }
    filename: { type: string }
    size: { type: integer, format: int64 }
```

```
            InitUploadResponse:
              type: object
              properties:
                upload_id: { type: string }
                offset: { type: integer, format: int64 }

            UploadIdRequest:
              type: object
              required: [upload_id]
              properties:
                upload_id: { type: string }

            CreateShareRequest:
              type: object
              required: [file_path, expires_in_secs]
              properties:
                file_path: { type: string }
                expires_in_secs:
                  type: integer
                  format: int64
                  description: Durée de validité, bornée entre 60 s et 1 an.
                password: { type: [string, 'null'], format: password }
                max_downloads:
                  type: [integer, 'null']
                  format: int64
                  description: Limite de téléchargements ; 1 pour un lien à usage unique.

            ShareLinkView:
              type: object
              description: Vue d'un lien de partage (le hachage du mot de passe n'est jamais
exposé).
              properties:
                token: { type: string }
                file_path: { type: string }
                filename: { type: string }
                created_by: { type: string }
                created_at: { type: string, format: date-time }
                expires_at: { type: string, format: date-time }
                has_password: { type: boolean }
                max_downloads: { type: [integer, 'null'], format: int64 }
                download_count: { type: integer, format: int64 }

            SharedFolderMember:
              type: object
              required: [username, can_write]
              properties:
                username: { type: string }
                can_write: { type: boolean }

            SharedFolder:
              type: object
              properties:
                id: { type: string }
                name: { type: string }
                path: { type: string }
                created_by: { type: string }
```

```
        created_at: { type: string, format: date-time }
        members:
          type: array
          items: { $ref: '#/components/schemas/SharedFolderMember' }

    CreateSharedFolderRequest:
      type: object
      required: [name, path]
      properties:
        name: { type: string }
        path: { type: string }

    InternalShare:
      type: object
      properties:
        id: { type: string }
        path: { type: string }
        name: { type: string }
        is_dir: { type: boolean }
        owner: { type: string }
        recipient: { type: string }
        can_write: { type: boolean }
        created_at: { type: string, format: date-time }

    CreateInternalShareRequest:
      type: object
      required: [path, username]
      properties:
        path: { type: string }
        username: { type: string, description: Nom d'utilisateur du destinataire. }
        can_write: { type: boolean, default: false }

    InternalSharesResponse:
      type: object
      properties:
        owned:
          type: array
          items: { $ref: '#/components/schemas/InternalShare' }
        received:
          type: array
          items: { $ref: '#/components/schemas/InternalShare' }

    Favorite:
      type: object
      properties:
        username: { type: string }
        path: { type: string }
        name: { type: string }
        is_dir: { type: boolean }
        created_at: { type: string, format: date-time }

    AddFavoriteRequest:
      type: object
      required: [path]
      properties:
        path: { type: string }
```

```
            UploadRequestView:
              type: object
              properties:
                token: { type: string }
                name: { type: string }
                folder_path: { type: string }
                created_by: { type: string }
                created_at: { type: string, format: date-time }
                expires_at: { type: [string, 'null'], format: date-time }
                has_password: { type: boolean }
                is_expired: { type: boolean }

            CreateUploadRequest:
              type: object
              required: [name, folder_path]
              properties:
                name: { type: string }
                folder_path: { type: string }
                expires_in_secs: { type: [integer, 'null'], format: int64 }
                password: { type: [string, 'null'], format: password }

            AuditEntry:
              type: object
              properties:
                timestamp: { type: string, format: date-time }
                username: { type: string }
                action: { type: string }
                resource: { type: [string, 'null'] }
                ip: { type: [string, 'null'] }
                details: { type: [string, 'null'] }

            TotpCode:
              type: object
              required: [code]
              properties:
                code: { type: string, description: Code TOTP à 6 chiffres. }

            TotpRecoveryRequest:
              type: object
              required: [recovery_code]
              properties:
                recovery_code: { type: string }

            TotpStepUpRequest:
              type: object
              description: >
                Confirmation renforcée exigée pour modifier une 2FA active (désactivation,
                régénération des codes de récupération).
              required: [password, code]
              properties:
                password: { type: string, format: password, description: Mot de passe
courant. }
                code:
                  type: string
                  description: Code TOTP courant à 6 chiffres, ou code de récupération
```

```
(consommé).

            TotpSetupResponse:
              type: object
              properties:
                qr_code_png_b64:
                  type: string
                  description: URI otpauth:// à afficher sous forme de QR code.

            TotpConfirmResponse:
              type: object
              properties:
                backup_codes:
                  type: array
                  items: { type: string }
```